Reinforcement Learning

ARI2204

Jake Desira (0023806L)

&

Tristin Bezzina (0176006L)

Table of Contents

Algorithms Overview & Implementation *1*

Overview 1

1. Monte Carlo (On-Policy Control) 3
2. SARSA (On-Policy Control) 4
3. Q-Learning / SARSAMAX (Off-Policy Control) 5
4. Double Q-Learning (Off-Policy Control) 6

Evaluation Methodology *7*

Appendix A: Distribution of Work *14*

Appendix B: Plagiarism Form *15*

Appendix C: Use of Generative AI *16*

Appendix D: References *18*

Algorithms Overview & Implementation

Overview

This project explores the use of **reinforcement learning** in a simplified version of Blackjack, with the aim of training an agent to learn an effective playing strategy through repeated interaction with the environment. Rather than relying on predefined rules alone, the agent improves its decisions over many episodes by observing the long-term outcomes of choosing to **HIT** or **STAND** in different situations. We will focus on comparing the methods used:

- **Monte Carlo**,
- **SARSA**,
- **Q-Learning**,
- **Double Q-Learning**.

Each of these algorithms approaches learning differently, especially in the way they balance **exploration** and **exploitation**.

Since the project required multiple agents that operate on the same Blackjack environment, an **object-oriented programming (OOP)** approach was adopted. Using OOP helped keep the code **modular**, improved **readability**, and reduced repetition through **code reusability**. To support this design, a BaseAgent.py class was introduced to hold the functionality that was common across all agents. This allowed the specialised agents to inherit shared behaviour instead of re-implementing the same logic multiple times. The *BaseAgent* class included these shared mechanisms:

- the **ε-greedy policy**
- the **count increment logic**
- the computation of **α (alpha)**, used as the learning rate
- helper methods for selecting the greedy action from a given state

The **ε-greedy policy** is especially important because it defines how the agent balances **exploration** and **exploitation**. In practice, this means that with probability **ε**, the agent selects a random action in order to explore alternatives, while with probability **1 − ε**, it selects the action with the highest current Q-value. This is referred to as "greedy" because the policy chooses the immediate best-known action according to the learned estimates.

Another important implementation decision concerned the representation of the **Q-table** and **count table**. These were stored using **nested dictionaries** rather than matrices. Dictionaries were considered superior primarily because they provide a **direct semantic mapping**, allowing states and actions to be used as actual keys rather than arbitrary integer indices. With a matrix, each row and column would only represent positions, meaning that an additional encoding or translation layer would be required to interpret which index corresponds to which state-action pair. In contrast, a dictionary structure such as *table[state][action]* makes the code largely self-documenting and significantly easier to

debug. This approach was chosen over a 2D matrix to prioritize code readability and ease of debugging. Given the relatively small state-action space (180 pairs), the computational overhead of dictionary lookups was negligible and did not impact the agent's performance

The project includes both **on-policy** and **off-policy** reinforcement learning methods. The key distinction between these two categories lies in what policy is being learned. In **on-policy** methods, the agent learns the value of the same policy it is currently following, meaning the behaviour policy and the target policy are effectively the same. In **off-policy** methods, however, the agent may behave according to one policy while learning the value of another, typically greedier, target policy. This difference is especially important when comparing how exploration affects learning stability and convergence.

A notable design choice in the environment implementation was the way each environment instance was initialised. Every new episode automatically begins in a valid Blackjack state, and the environment logic ensures that the game progresses correctly through the **non-learned decision region**. In particular, if the player's total is **less than 12**, the sensible action is always to **HIT**, whereas if the total is **21**, the player should always **STAND**.

Special attention was also given to the handling of the **usable Ace** condition. A Boolean flag was used to indicate whether the player's hand currently contains an Ace being counted as **11** without causing the hand to bust. This distinction is necessary because two hands with the same total can have very different strategic meanings depending on whether the total is "soft" or "hard". For example, a hand total of 17 with a usable Ace is less risky than a hard 17, since the Ace can effectively revert to a value of 1 if needed. In addition to the usable Ace flag, the environment also makes use of **reward** and **done** flags. The **reward** flag communicates the numerical outcome of a transition or terminal state, while the **done** flag indicates whether the current episode has ended. These are essential because the agents must know both the immediate feedback from an action and whether further interaction should continue.

To make the public interface of the environment clearer, some helper methods were intentionally marked as **private** using the __ prefix. Although Python does not provide strict encapsulation in the same way as some other languages, this naming convention helps communicate which methods are intended for internal use only. This improves readability by communicating clearly which methods form part of the environment's usable external interface, and which exist only to support its internal mechanics such as hand valuation, card drawing, or dealer behaviour.

From a practical development perspective, teamwork was coordinated using **Git** and a shared **GitHub repository**. This made collaboration significantly more efficient. Using a repository also made it easier to resolve conflicts and maintain a shared project history. This was particularly useful for a group project where the work was divided across different files and responsibilities. Furthermore, because the project relied on external plotting functionality, a separate development environment was configured in **VS Code** to manage the dependency **Matplotlib** cleanly.

1. Monte Carlo (On-Policy Control)

The first algorithm implemented was **Monte Carlo**. In contrast to temporal-difference methods, Monte Carlo does **not** update the Q-values immediately after each action. Instead, it waits until the **entire episode has terminated**, observes the final outcome of the round, and then updates the visited state-action pairs using that return.

A dedicated MonteCarloAgent class was used for this algorithm. This class inherits the common functionality from *BaseAgent*, but overrides the action-selection logic in order to support the two Monte Carlo variants required **Exploring Starts** and **No Exploring Starts**

For the **Exploring Starts** variant, when the first state of the episode that falls in the learning range [12,20] is encountered, the action is forced to be random. After that point, subsequent actions in the same episode follow the normal ε-greedy rule. This was implemented using a Boolean flag that tracks whether the current learned decision is the **first one** of the episodes. For the **No Exploring Starts** variant, no such forced random first move is used but instead, every learned decision is selected according to the ε-greedy policy immediately.

Another important aspect of the implementation is that the Monte Carlo agent does **not** update Q-values inside its *run_episode()* method. Instead, its responsibility is to generate one complete episode and return two things:

- the **episode trace**, i.e. the list of visited (state, action) pairs
- the **final reward** of the episode

The algorithm must wait until the end of the episode before learning from the actual return. During the episode, whenever the agent visits a learned state in the range [12,20], the current *(state, action)* pair is appended to an *episode_trace*. Once the episode terminates, the final reward is known, and the calling code in Evaluation.py iterates through that trace to update the Q-values.

The Q-value update itself follows the standard **incremental Monte Carlo** form. After an episode ends, each visited state-action pair in the trace is updated towards the final reward G, which in this Blackjack setting is simply the terminal outcome of the round. Since an incremental form was used, the update rule does not need to store all past returns explicitly. Instead, the count table keeps track of how many times each *(s,a)* pair has been encountered, and this count is used to derive the step size. In practical terms, the Q-value is gradually moved towards the newly observed return, rather than being replaced outright.

The version implemented in this project corresponds to an **every-visit Monte Carlo** approach. This means that if the same state-action pair appears multiple times within a single episode trace, each occurrence is updated after the episode terminates. This was chosen because it integrates naturally with the current episode-trace structure and avoids the additional bookkeeping that would be needed to filter only first visits. Given the short episodic nature of Blackjack, this approach was sufficient and straightforward to implement.

2. SARSA (On-Policy Control)

Unlike Monte Carlo, the following algorithms are temporal-difference methods hence this means they are updates Q-values immediately after each learned transition. This makes them **bootstrap** methods, because the update for the current state-action pair is based not on the final outcome of the episode, but on the current estimate of the best possible action in the next state. As a result, learning happens continuously throughout the episode rather than only after termination.

For **SARSA**, however, the update does not assume that the best possible action will be taken in the next state. Instead, it depends on the action that is **actually selected** by the behavioural policy in the successor state. This is because SARSA is an **on-policy** temporal-difference algorithm, meaning that the same ε-greedy policy is used both to interact with the environment and to evaluate the update. As a result, the learning process naturally follows transitions of the form:

$$(state, action) \rightarrow (next_{state}, next_{action})$$

since the update uses the action actually taken in the successor state rather than the greedy action from the Q-table, as done in Q-Learning and Double Q-Learning.

In code, SARSA was implemented in a dedicated SarsaAgent class that inherits the shared functionality from BaseAgent, such as ε-greedy action selection, count increment logic, and learning-rate calculation. The main learning logic was placed inside *run_episode()*, where the initial state and action are selected before entering the loop. This design makes each iteration correspond clearly to one complete transition of the form (S, A, R, S', A'), which improves readability and follows the standard presentation of SARSA. At each step, the selected action is applied to the environment, which returns the next state, reward, and terminal flag. If the episode has not ended, the next action is selected immediately from the successor state using the same ε-greedy policy. The visit count for the current state-action pair is then incremented, and the Q-value is updated using the observed reward together with the Q-value of the actual next action.

Additionally, in this implementation of SARSA, it was necessary to explicitly handle cases where the episode ended before the agent could select an action. This occurs because the environment only returns learning states when the player sum is between **12 and 20**. If the player automatically busts or reaches **21** due to the hard-coded rules applied before learning begins, the environment returns no valid learning state. Therefore, this condition had to be checked before entering the update loop, since otherwise the algorithm would attempt to select an action from a state that does not exist.

3. Q-Learning / SARSAMAX (Off-Policy Control)

The specialised behaviour of Q-Learning is concentrated mainly in its update rule and episode execution logic, while the more general functionality remains in the parent class. This makes the implementation more compact and easier to compare with the other agents.

Conceptually, Q-Learning is an **off-policy** algorithm because the policy used to **behave** is not exactly the same as the policy being **learned**. The agent still follows an ε-greedy behaviour policy when choosing actions. However, when updating the Q-value, it does not use the next action that the behaviour policy actually selected. Instead, it assumes that from the next state onward, the best possible greedy action will be taken. In other words, the algorithm learns the value of the **greedy target policy**, even while behaviour is still partially exploratory.

This difference is reflected directly in the update rule. After the agent executes an action and transitions to the next state, the Q-value of the current state-action pair is updated toward:

- the **immediate reward**, plus
- the **maximum estimated Q-value available in the next state**

This is what distinguishes Q-Learning from SARSA. Q-Learning updates using the **best possible next action** according to the current Q-table. This generally makes Q-Learning more optimistic, since it assumes future behaviour will be greedy even while exploration is still taking place.

A key part of the Q-Learning implementation is the treatment of **terminal states**. If the selected action causes the episode to end, for example because the player busts or the dealer finishes resolving the round after a stand, then there is no future state from which to bootstrap. In such cases, the update target reduces to just the terminal reward. Otherwise, the algorithm retrieves the highest available Q-value from the next state and uses it as the bootstrapped estimate of future return. This distinction is important, since continuing to bootstrap from a terminal state would produce incorrect updates.

Like the other temporal-difference algorithms, Q-Learning uses the count table to derive the learning rate. Each time a learned state-action pair is encountered, its count is incremented, and the corresponding step size decreases as that pair is visited more often. This means the algorithm learns aggressively when a pair is still relatively new, but becomes more stable as more experience is gathered

4. Double Q-Learning (Off-Policy Control)

Double Q-Learning solves a problem that the standard Q-Learning algorithm faces, called **maximisation bias**. This happens because the same Q-table is used both to choose the best action and to evaluate that action while still following the same overall update workflow conceptually. Due to this, noise in the estimates causes Q-values to become over optimistic, since the algorithm repeatedly selects the maximum value from the same table that it is updating. Double Q-Learning addresses this limitation by keeping two independent Q-tables, where each table helps update the other instead of updating itself directly. This ensures that the same estimates are not used both for selecting the successor state-action pair and evaluating the current state-action pair.

Hence, because of this separation between selection and evaluation, instead of selecting the best action at the successor state based on the Q-values from the same table where the Q-value of the current state-action pair exists, the algorithm makes use of **another separate Q-table** to evaluate the update, which helps prevent maximisation bias. In order to keep both Q-tables valid throughout the entire learning process both of them need to be updated and have Q-values which eventually converge. However, they can't be modified at the same time because it would lead us back to the problem Q-Learning faces. Hence why the implementation consists of some random generated and if it exceeds the threshold 0.5, q-table is updated, otherwise *q-table_b* is updated. If q-table is selected for updating, the best action at the successor state is selected using q-table, but the value of that action is evaluated using *q-table_b*, and vice versa. This ensures that both value estimates contribute equally to decision making while remaining independent.

Another important part of the Double Q-Learning algorithm is the behavioural policy used for action selection. Instead of selecting actions using only one Q-table, the algorithm chooses actions based on the average Q-value from both tables. Formally defined as follows:

$$\underset{a}{argmax}\left(\frac{Q_1(s,a)+Q_2(s,a)}{2}\right)$$

This ensures that action selection reflects information learned from both tables equally and prevents the policy from depending too heavily on only one value estimate.

Like all the other algorithms, a learning rate α is used so that the correction applied when updating the current state-action value is not too large. As required, this learning rate was based on how many times each state-action pair was visited during the entire number of episodes. If a pair was visited only a few times, then it is still far from convergence, therefore the step size should allow the correction to have a larger effect on the current value. However, if a pair was visited many times, then it is already close to convergence, and the correction should be smaller, since a large update at that stage might be caused by a one-off scenario rather than representing the true expected value. In this case, the correction refers to the difference between the update target and the current estimate, which is computed as the immediate reward plus the successor state-action value minus the current state-action pair value.

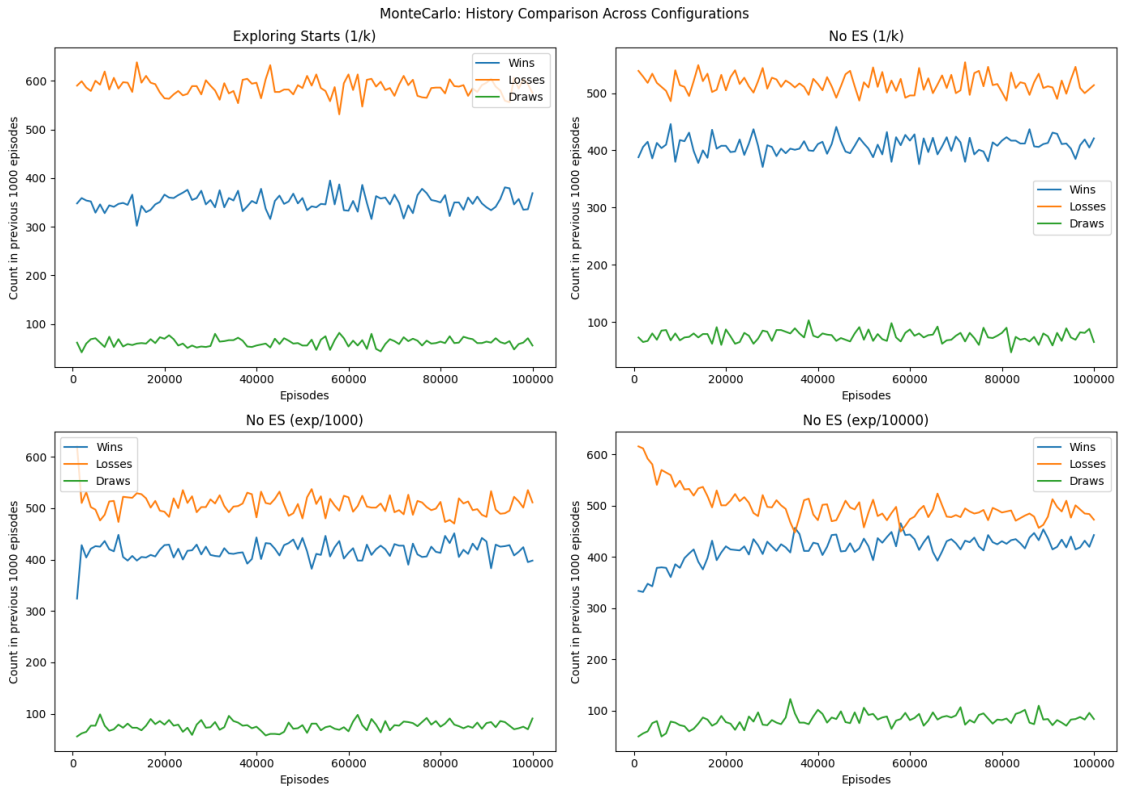
Evaluation Methodology

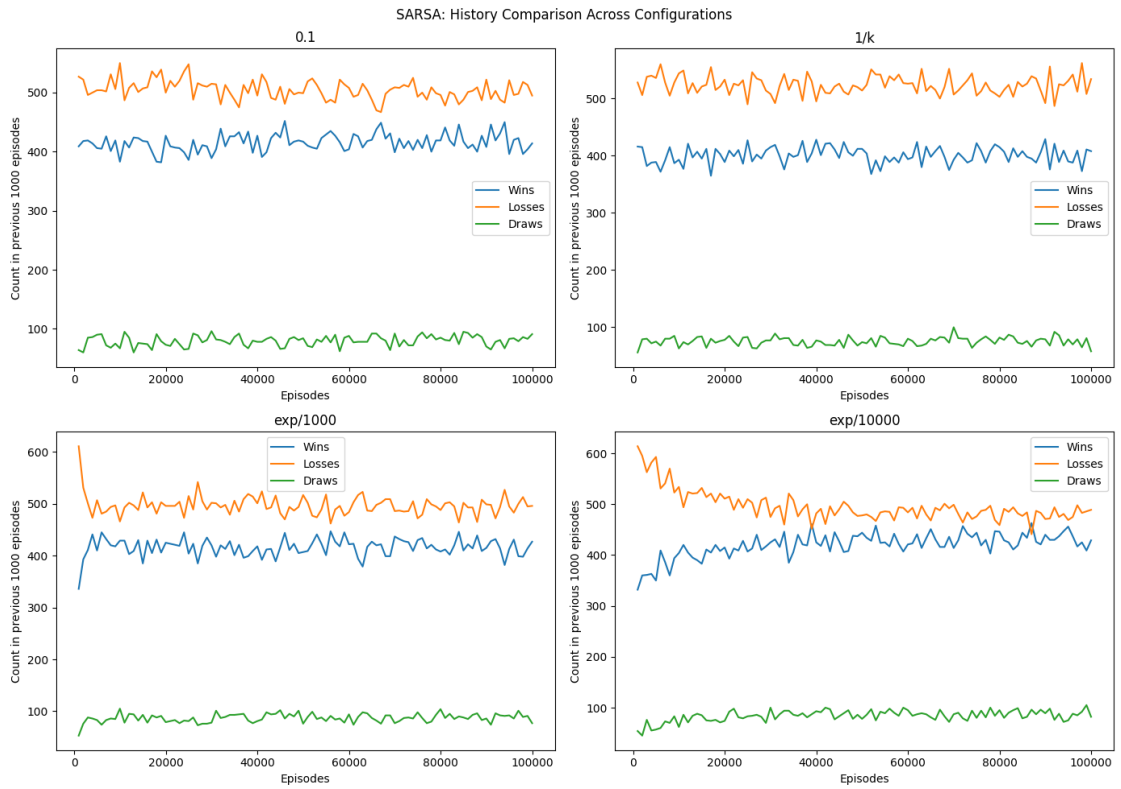
When interpreting the results, it is important to note that the dealer is expected to maintain some advantage in this Blackjack setting. Since the player acts first, the player may lose immediately by going over 21 before the dealer is required to take any further action. Because of this, it is normal for the number of **losses** to remain higher than the number of **wins**, even when learning is successful. For this reason, the focus of the evaluation is not simply on whether wins exceed losses, but on whether a configuration shows **improvement over time**, **better final statistics**, and a **lower dealer advantage** than alternative exploration settings.

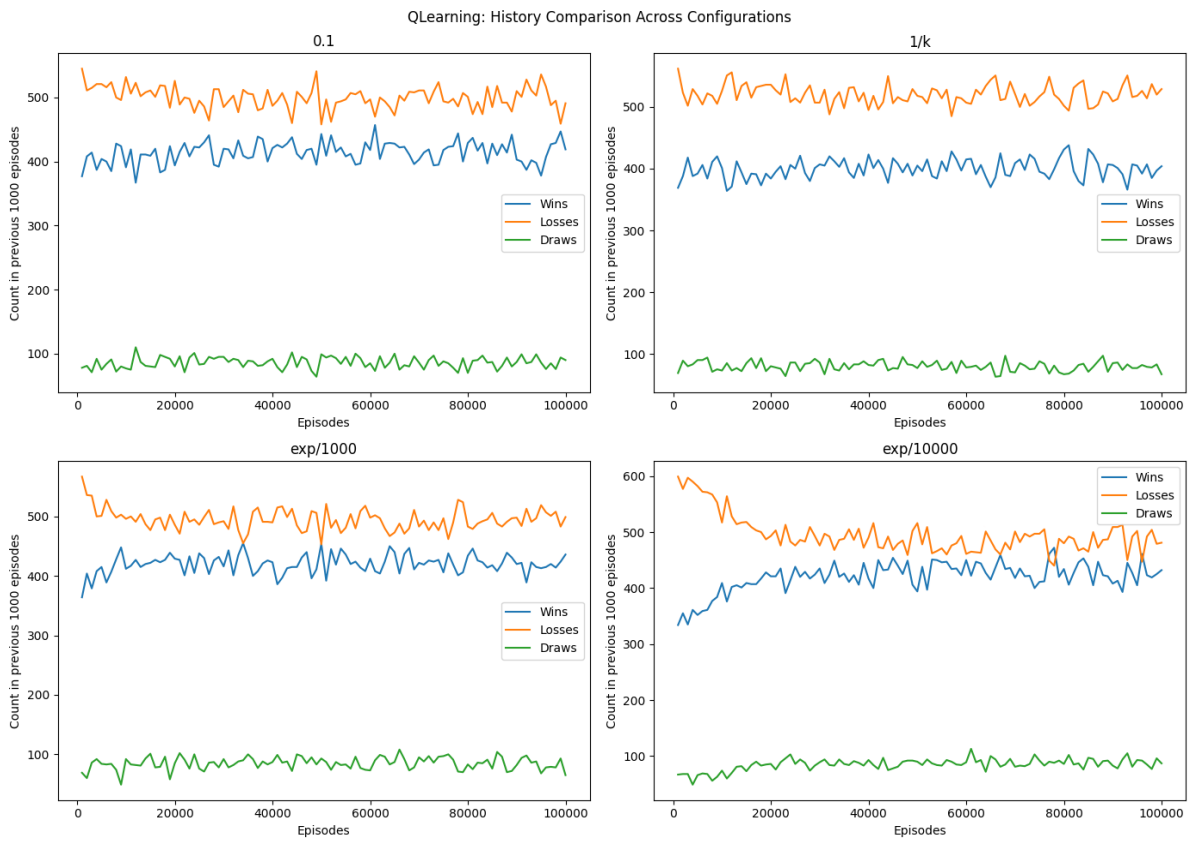
Across all four algorithms, the effect of the exploration configuration followed a broadly similar pattern. The weakest-performing settings were generally those where exploration was either too disruptive or reduced too quickly. A **fixed** $\epsilon = 0.1$ often produced stable but limited improvement, since the agent continued to explore at the same rate even in the later stages of training, **preventing the policy from fully settling**. The $\frac{1}{k}$ configuration also tended to underperform, as exploration decayed very quickly, which likely caused the agents to become **overly greedy** before they had explored the state space sufficiently. In contrast, the exponential schedules produced stronger results overall, particularly $e^{-\frac{k}{10,000}}$, which consistently gave the best or near-best performance. This slower decay **kept exploration active** for longer, allowing the agents to continue discovering useful actions early in training while still gradually shifting towards exploitation later on. As a result, configurations using $e^{-\frac{k}{10,000}}$ generally showed clearer reductions in losses and stronger increases in wins. For Monte Carlo, the **Exploring Starts** configuration performed worst overall. In Blackjack, this is especially significant because the first forced random action can be either **HIT** or **STAND**; if the random action is **STAND**, the **round ends immediately**, which shortens many episodes and reduces the chance of visiting deeper learned state-action pairs. This makes the exploration less balanced than in the other stochastic configurations, where random behaviour is distributed more naturally across training.

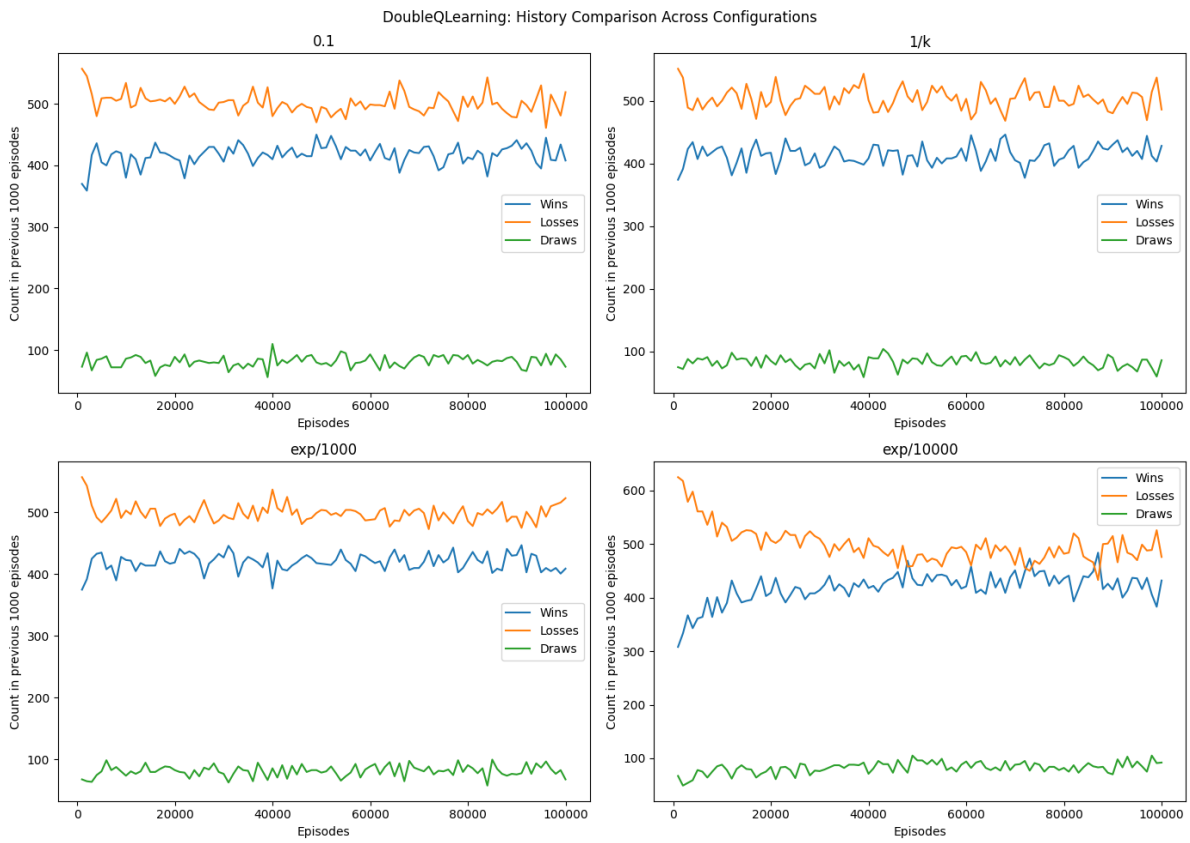
Although the algorithms differ in how they update their state-action values internally, the plots show that their overall performance remains broadly similar. This is not necessarily surprising in the present Blackjack setting, because the problem has a relatively **small and well-defined decision space**, the reward structure is simple, and all methods are ultimately learning over the same set of states, actions, and episode outcomes. In other words, while the intermediate learning process differs across algorithms, they are still being guided towards a similar set of sensible Blackjack decisions. As a result, the main visible differences in the graphs come less from the algorithm family itself and more from **how exploration is controlled**. This is why the curves across algorithms often settle into similar ranges, while the exploration schedule has a clearer effect on whether learning

improves steadily, stabilises too early, or remains too random. Therefore, in this task, it is more meaningful to compare **how each configuration shapes learning behaviour** than to argue that one algorithm is decisively superior to another based only on these performance curves.

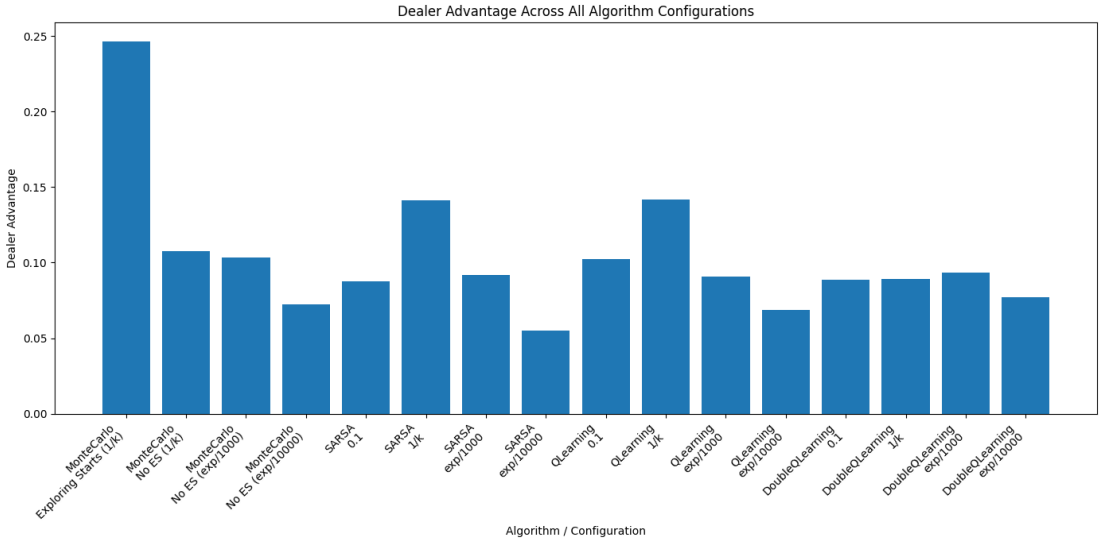




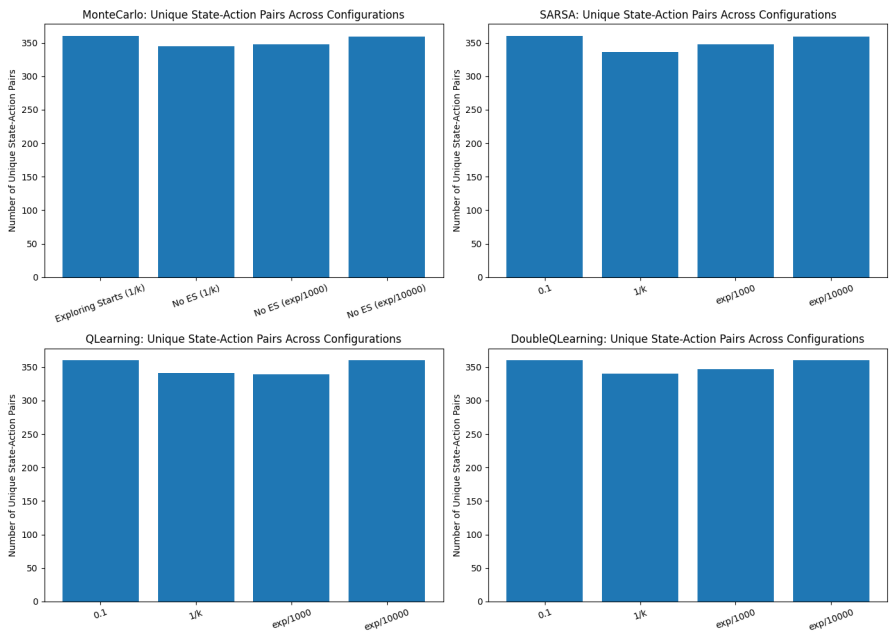

QLearning: History Comparison Across Configurations
0.1
1/k
exp/1000
exp/10000
Count in previous 1000 episodes
Episodes
Wins
Losses
Draws


DoubleQLearning: History Comparison Across Configurations
0.1
1/k
exp/1000
exp/10000
Count in previous 1000 episodes
Episodes
Wins
Losses
Draws

The differences between exploration schedules are further confirmed by the dealer-advantage comparison across all algorithm configurations. Lower dealer-advantage values indicate stronger learned policies because they reflect fewer losses relative to wins. As shown in the figure, configurations using exponential decay particularly $e^{-\frac{k}{10{,}000}}$ consistently achieved some of the lowest dealer-advantage values across Monte Carlo, SARSA, Q-Learning, and Double Q-Learning. In contrast, configurations with fixed exploration or rapidly decaying exploration generally produced higher dealer-advantage values, supporting the earlier observation that insufficiently balanced exploration limits policy quality.



The **unique state-action pair** charts reinforce this interpretation. Across all algorithms, the total number of explored state-action pairs remains within a relatively narrow range, which suggests that the different methods ultimately covered a broadly similar portion of the Blackjack state space. This again supports the idea that the problem is limited enough for multiple algorithms to reach comparable levels of coverage.



In the **no usable Ace** tables, all four algorithms show a clear preference for **STAND** on high player sums, especially from **17 to 20**, while lower totals such as **12 to 15** are more sensitive to the dealer's visible card. In those weaker hands, the policy becomes more mixed, with **HIT** appearing more often when the dealer shows a stronger card, which is consistent with sensible Blackjack play. In the **usable Ace** tables, the policies are slightly more aggressive, with more **HIT** decisions appearing at totals that would be riskier without a soft hand. This also makes sense, since a usable Ace reduces the danger of busting and allows the player to take more risks.

As discussed, we can expect that all algorithms display a very similar overall structure, particularly in the stronger regions of the policy where the best action is more obvious. However, some individual state decisions still appear less intuitive, which suggests that the policies have not converged perfectly in every state. For example, in some cases a player sum of **12** against a dealer **10** or **Ace** is mapped to **STAND**, even though this would usually be expected to favour **HIT** in standard Blackjack reasoning. This likely reflects the fact that both actions are relatively poor in such states, so the learned policy selects the one with the slightly better sampled expected return, rather than always matching the intuitively strongest human choice. It also suggests that a few borderline states had not been explored thoroughly enough under the final stages of training. In other words, while the overall policy became increasingly exploitative, the exploration schedules may have decayed too far for these less stable states to continue testing alternative actions often enough. Therefore, although the algorithms appear to converge towards a similar practical strategy, the tables suggest that this convergence is not perfectly optimal, and that some edge-case decisions remain influenced by limited late-stage exploration, sampling noise, or incomplete refinement of the Q-values.

MonteCarlo - No ES (exp/10000) - No Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	H
16	S	S	S	S	S	S	H	S	H	H
15	S	S	S	S	S	S	H	H	S	H
14	S	S	S	S	S	S	H	H	H	H
13	S	S	S	S	S	H	H	H	H	H
12	H	S	S	S	H	H	H	H	H	H

MonteCarlo - No ES (exp/10000) - Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	H	S	H	S	H	S	S	H	S	H
17	H	H	H	S	S	H	H	H	H	H
16	H	H	H	S	H	H	H	H	H	H
15	H	H	H	H	H	H	S	H	H	H
14	S	H	S	H	H	H	H	H	H	H
13	H	S	S	S	S	H	H	H	H	H
12	S	H	H	S	H	S	S	H	H	H

SARSA - exp/10000 - No Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	H
16	S	S	S	S	S	S	H	S	S	H
15	S	S	S	S	S	H	S	H	S	H
14	S	S	S	S	S	H	H	S	H	H
13	S	H	S	S	S	S	H	S	H	H
12	S	H	S	S	S	H	H	H	H	H

SARSA - exp/10000 - Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	H
18	H	H	H	S	H	S	H	S	H	H
17	S	H	H	S	H	H	S	H	H	S
16	H	H	H	H	H	H	H	H	H	H
15	H	H	H	H	H	H	H	H	H	H
14	H	S	S	S	H	H	H	H	H	H
13	H	H	H	H	H	H	H	H	H	H
12	S	H	H	H	H	H	H	H	H	H

QLearning - exp/10000 - No Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	H
16	H	S	S	S	S	H	H	H	S	H
15	S	S	S	S	S	H	S	H	H	H
14	S	S	S	S	S	H	H	S	H	H
13	S	S	S	S	S	H	H	H	H	H
12	H	S	S	H	S	H	H	H	H	H

QLearning - exp/10000 - Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	H	H	H	S	H	S	S	S	S	H
18	S	S	H	H	S	S	S	H	H	H
17	H	S	H	S	S	H	H	H	H	H
16	H	H	H	H	H	H	H	H	H	H
15	H	H	H	H	H	H	H	S	H	H
14	H	H	H	H	H	H	H	H	H	H
13	H	H	H	H	H	H	H	H	H	H
12	H	H	H	H	H	H	H	H	H	H

DoubleQLearning - exp/10000 - No Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	H
18	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	S
16	S	S	S	S	S	S	H	S	S	S
15	S	S	S	S	S	S	H	S	S	S
14	S	S	S	S	S	S	H	S	S	S
13	S	S	S	S	H	S	S	S	S	S
12	S	S	S	S	S	H	S	S	S	S

DoubleQLearning - exp/10000 - Usable Ace

Player Sum	2	3	4	5	6	7	8	9	10	A
20	S	S	S	S	S	S	S	S	S	S
19	S	S	S	S	S	S	S	S	S	S
18	H	H	H	S	S	S	S	S	S	H
17	S	S	S	H	S	H	S	S	H	S
16	H	H	H	H	S	H	H	H	H	H
15	H	H	S	H	H	H	S	H	S	S
14	H	H	H	H	H	S	H	S	S	S
13	H	H	H	H	H	H	H	H	H	H
12	H	H	S	H	H	H	H	H	H	H

Appendix A: Distribution of Work

Jake Desira

- Implemented the **Monte Carlo** and **Q-Learning** algorithms
- Wrote the related **documentation** for these algorithms
- Implemented Environment.py
- Implemented Plots.py

Signature: _____

Tristin Bezzina

- Implemented the **SARSA** and **Double Q-Learning** algorithms
- Wrote the related **documentation** for these algorithms
- Implemented BaseAgent.py
- Implemented Evaluation.py

Signature: _____

Appendix B: Plagiarism Form

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

Jake Desira
Student Name

Signature

Tristin Bezzina
Student Name

Signature

ARI 2204
Course Code

Reinforcement Learning BlackJack Group
Title of work submitted

11/04/2026
Date

Appendix C: Use of Generative AI

AI Tool Used - ChatGPT (GPT-5.4 Thinking / Auto mode)

Assistance

The tool was used to support:

- clarification of reasoning behind the **evaluation plots**
- checking and refining **code structure** where sometimes it suggested better code structure
- support for structuring Plots.py, especially when working with the **Matplotlib** library
- organising and refining the **written documentation** of the project, particularly improving the wording and structure of written sections without replacing our original ideas

The tool was **not** used to replace the group's own problem-solving, implementation, or conclusions. It was mainly used as a support tool for checking reasoning, improving structure, and refining presentation.

Example Prompts Used

The following are representative prompts used during the project:

- "Does this strategy-table decision make sense, or could it be caused by incomplete convergence or sampling noise?"
- "Can different reinforcement learning algorithms show very similar final win/loss/draw curves in Blackjack even if they learn differently internally?"
- "Can you help me identify why this Python error is happening in my Plots.py?"
- "Is it true that the algorithms converged in the region between 20k to 30k episodes in this plot?"
- "Explain thoroughly how this implementation of BlackJack should work"
- "Do you think that it's better to have a fixed Q and count table or to have them dynamically built?"
- "Given these ideas....re-write it so that it's more understandable for the report"

Verification of AI Responses

All AI-assisted output was treated with caution and was **manually verified** before use. In several cases, ChatGPT produced responses that were incomplete, imprecise, or not

fully aligned with the project brief or the actual code implementation. Because of this, the following checks were applied:

- algorithm explanations were verified against the **project brief**, the implemented code, and the group's own understanding
- plotting suggestions were checked by actually running the code and confirming that the output matched the required evaluation metrics
- explanations of reinforcement learning behaviour were compared against the group's own results and against the logic of the implemented algorithms
- any written text suggested by AI was reworded and adapted so that it reflected the group's actual implementation and interpretation
- where AI suggested something that conflicted with the brief, we opted with the brief.

In practice, hallucinations and inaccurate suggestions were encountered multiple times, especially when discussing convergence, interpretation of borderline Blackjack states, and assumptions about what the repo or plots showed. These responses were discarded or corrected after manual checking.

Appendix D: References

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] A. Kumar, “SARSA Algorithms Explained,” Medium, Nov. 2024. [Online]. Available: https://medium.com/@heyamit10/sarsa-algorithms-explained-b94078fd658b848.

[3] Invisible AI Guru Jii, “Monte Carlo Methods in Reinforcement Learning,” Medium, Nov. 9, 2024. [Online]. Available: https://medium.com/@apukumargiri1/monte-carlo-methods-in-reinforcement-learning-04a8e406b848.

[4] P. Dahal, “Monte-Carlo RL Methods,” Sept. 30, 2021, updated Jul. 15, 2025. [Online]. Available: https://parasdahal.com/notes/monte-carlo%2Brl%2Bmethods/.

[5] J. Bajada, “Reinforcement Learning Lecture Notes,” University of Malta, 2026. Unpublished course notes.